

Competitor Price and Availability Intelligence Platform

An end-to-end pipeline that tracks competitor prices across retailers, records every change, forecasts short-term movement, and alerts on undercutting.

SUBMITTED BY Love Vyas	DATE 1 September 2026	DOMAIN Retail price intelligence
CORE STACK Python, Celery, PostgreSQL, dbt	DATA SOURCE Open Prices (public API)	SCALE DELIVERED 8,433 price events

Contents

1. Problem statement

2. Objectives and scope

3. Data sources

4. Solution architecture

5. Technology stack

6. Database design

7. Data pipeline

8. Analytics layer
9. Forecasting

10. Product matching

11. Serving and visualization

12. Infrastructure

13. Testing and data quality

14. Results

15. Challenges and resolutions

16. Limitations and future work

Section 1 — Problem Statement

Business context

Retailers and brands lose margin in two ways that are invisible without continuous monitoring. A competitor quietly drops a price and takes volume before anyone notices; or a competitor goes out of stock and the demand that would have gone to them is never captured. Commercial tools such as Prisync, Competera and Minderest sell exactly this capability — competitor price monitoring, repricing signals, and market intelligence.

The problem

Build a system that continuously monitors competitor prices and availability across multiple retailers, records the full history of every change rather than only the latest value, and turns that history into decisions a category manager can act on: who is undercutting us and by how much, which products are volatile enough to need watching, and where prices are likely to move next.

The system must work from public data sources, must not lose history when a source is unavailable, and must be honest about the confidence of what it reports — a price observed three weeks ago should not be presented with the same authority as one observed today.

Expected outcome

A working pipeline that converts raw competitor price observations into a queryable analytical layer, a set of business marts answering specific pricing questions, short-term per-SKU forecasts with measured accuracy, an alerting path for undercutting, and a reporting layer suitable for both technical and non-technical consumers.

Section 2 — Objectives and Scope

OBJECTIVE	SUCCESS CRITERION	STATUS
Multi-source ingestion	Pluggable source interface; adding a source is one file	MET
Full change history	Every price and attribute change recorded, never overwritten	MET
Reproducibility	Warehouse rebuildable from raw payloads with no API calls	MET
Tested transformations	Business rules defined once and covered by data tests	MET
Forecasting	Per-SKU forecast with backtested error, benchmarked against a naive baseline	MET
Undercut alerting	Alerts raised with severity and evidence age, deduplicated	MET
Availability tracking	Out-of-stock frequency per SKU and retailer	BLOCKED
Cloud deployment	Infrastructure as code for the full stack	WRITTEN, NOT APPLIED

In scope: ingestion from public APIs, change data capture, warehouse modelling, forecasting, alerting, product matching, and reporting.

Out of scope: web scraping, checkout or transactions, multi-tenancy, and sub-minute streaming. No personal data is collected; the system holds public product prices only.

Section 3 — Data Sources

The platform is built on **Open Prices**, the price project of Open Food Facts. It is a public, keyless API carrying crowd-sourced retail prices, each with a real observation date and an OpenStreetMap store location. This matters: one product barcode genuinely appears at several named retailers over time, which is the exact shape a competitor-price platform needs. Product attribute data (name, brand, category) comes from the Open Food Facts catalogue embedded in the same payload.

SOURCE	AUTH	ROLE	STATUS
Open Prices / Open Food Facts	keyless	All real price, retailer and product data	LIVE
Fake Store API	keyless	Deterministic fixture for tests and local development	LIVE
Best Buy	API key	Planned. Publishes stock, which activates availability tracking	INTERFACE READY
eBay Browse	OAuth	Planned. Configured with the documented 5,000/day quota	INTERFACE READY
Digi-Key	OAuth	Planned. Publishes real stock and price tiers	INTERFACE READY

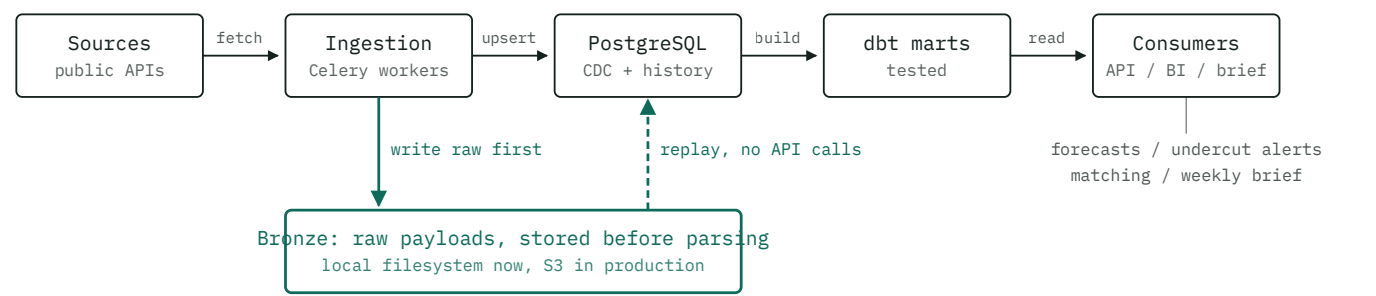
Data characteristics

PROPERTY	VALUE	IMPLICATION FOR DESIGN
Price observations	8,433	Partitioning needed for query performance
Distinct products	3,932	Per-SKU modelling is feasible
Distinct retailers	157	Genuine multi-retailer comparison possible
Distinct currencies	24	Currency must be part of the grain (see Section 8)
Observation span	2010-07-08 to 2026-09-01	Historical backfill must not be treated as current
Observations in last 30 days	3,736	Recent window drives forecasting and alerting

One authored dataset. Every price, product, retailer and date is real and comes from the API. The single exception is `own_catalog`, which represents the company's own catalogue — the thing competitor prices are compared against. No public API can supply that, so it is generated from products actually observed in the last 60 days, with our price set around each observed market average. Its provenance is documented alongside it.

Section 4 — Solution Architecture

Six stages, each independently runnable and inspectable. The defining design choice is that raw API payloads are persisted *before* parsing, which makes the database a derived artifact rather than the system of record.



Solid = normal flow. Dashed = recovery path.

Figure 1. Solution architecture in six stages. Raw payloads are written before parsing, so the warehouse can be rebuilt from them without contacting any source API.

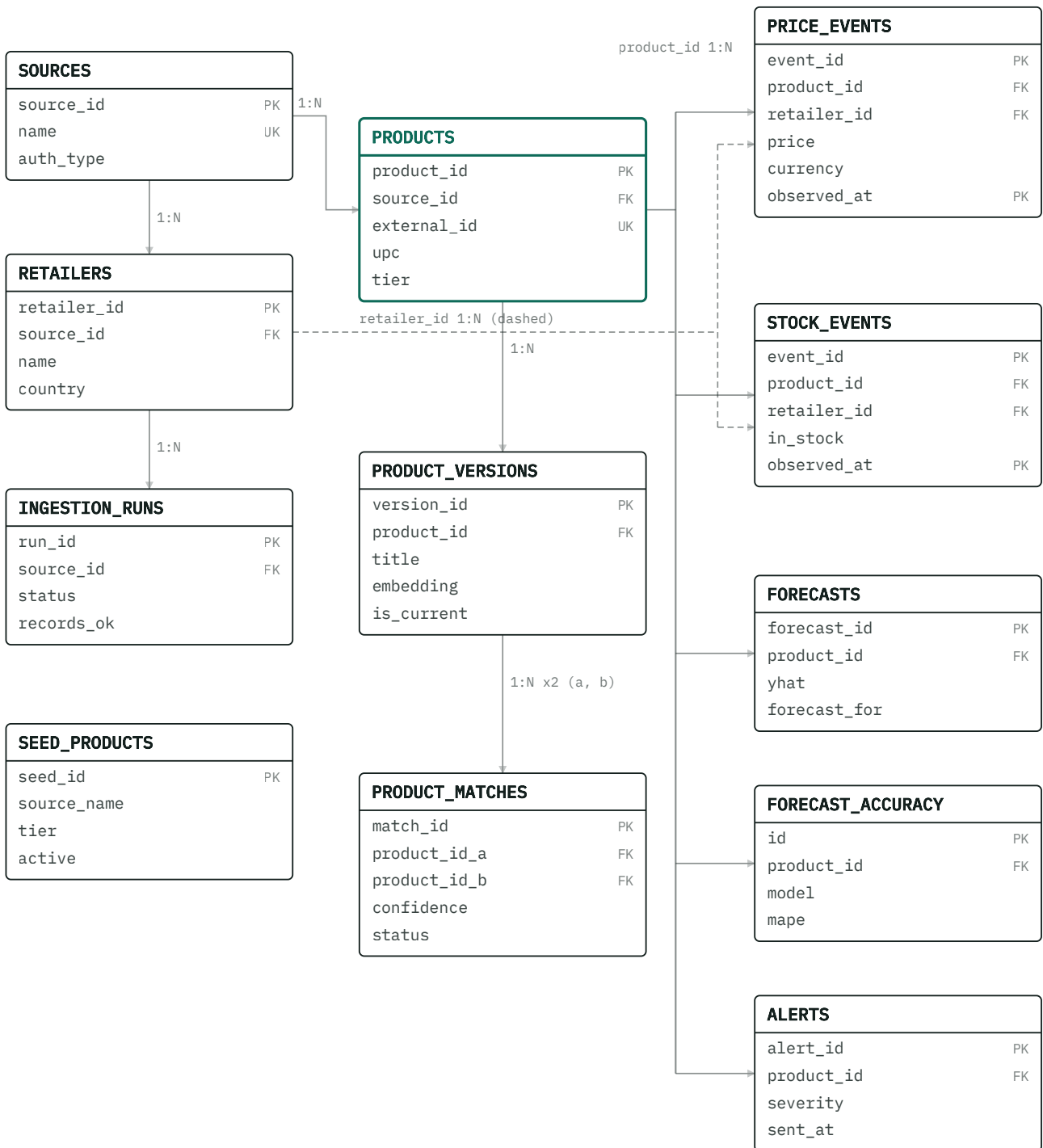
LAYER	RESPONSIBILITY
Ingestion	Scheduled fetch with per-source rate limiting and daily quota guards; retries with backoff; dead-letter for permanent failures
Bronze (raw zone)	Immutable gzipped JSON, partitioned by source and date, keyed by an idempotency hash
Serving database	Normalized reference data, slowly-changing dimension, append-only event tables
Transformation	dbt models in three layers, with data tests that gate the build
Analytics	Forecasting, product matching, undercut evaluation
Presentation	REST API, Metabase dashboard, Slack alerts, weekly written brief

Section 5 — Technology Stack

LAYER	TECHNOLOGY	REASON FOR SELECTION
Orchestration	Celery 5.5 + Celery Beat	Periodic polling with task fan-out; far lighter than a full orchestrator for this workload
Broker	Redis	Also backs the shared rate limiter, so one component serves two needs
Database	PostgreSQL 16 + pgvector	Partitioning, JSONB and vector similarity in one engine; no separate vector store
Transformation	dbt Core	SQL-first, testable, documented lineage
Validation	Pydantic v2 + Pandera	Per-record shape and cross-record batch contracts
Forecasting	Nixtla statsforecast	Fast per-SKU statistical models with native prediction intervals
Embeddings	fastembed (ONNX)	Same 384-dim model as sentence-transformers without the PyTorch dependency
API	FastAPI	Typed request and response models, generated OpenAPI documentation
Dashboard	Metabase	Connects directly to the marts; no BI-specific modelling layer needed
Review UI	Streamlit	Minimal code for an internal human-in-the-loop screen
Infrastructure	Terraform	Reviewable, version-controlled cloud definition

Section 6 — Database Design

Twelve tables in three groups: reference data, append-only event history, and derived output. The design separates *what a product is* (slowly changing) from *what was observed about it* (append-only), which is what makes exact change history possible.



PK primary key · FK foreign key · UK unique · dashed = retailer_id foreign key

price_events and stock_events are range-partitioned by month, so observed_at forms part of their primary key

Figure 3. Database structure. `products` is the hub: every table in the right-hand column carries `product_id` as a foreign key, drawn here as a single bus rather than five crossing lines. The two event tables are append-only and partitioned; `product_versions` holds slowly-changing attribute history with exactly one open row per product.

Table reference

TABLE	GROUP	GRAIN — ONE ROW PER	ROWS
<code>sources</code>	reference	upstream data source	2
<code>retailers</code>	reference	retailer within a source	157
<code>products</code>	reference	product within a source	3,932
<code>seed_products</code>	reference	SKU selected for scheduled tracking	60
<code>product_versions</code>	dimension (SCD2)	version of a product's attributes	3,932
<code>price_events</code>	fact	observed price per product, retailer and time	8,433
<code>stock_events</code>	fact	observed availability	0
<code>product_matches</code>	derived	candidate pair of equivalent products	2,557
<code>forecasts</code>	derived	predicted price for one product and date	77
<code>forecast_accuracy</code>	derived	backtested error per product and model	27
<code>alerts</code>	derived	alert raised, with delivery timestamp	62
<code>ingestion_runs</code>	operational	one ingestion cycle	—

Invariants enforced by the schema

These are constraints in the database, not conventions in application code, so no bug can violate them.

INVARIANT	MECHANISM
Exactly one open version per product	Partial unique index on <code>product_versions(product_id) where is_current</code>
No duplicate observations	Unique index on <code>(idempotency_key, observed_at)</code> ; retries and replays collapse
A match is an unordered pair	Check constraint <code>product_id_a < product_id_b</code>
Events remain queryable at volume	Monthly range partitions on <code>observed_at</code> , created on demand (115 in use)

Section 7 — Data Pipeline

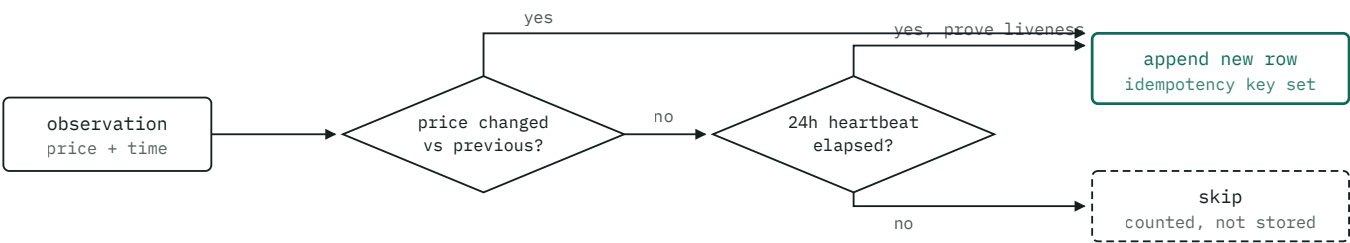
Extraction

Celery Beat enqueues per-source fetch tasks on three cadences: high-volatility SKUs every six hours, mid-tier daily, long-tail weekly. Before any HTTP call, the worker passes two Redis-backed guards — a token bucket for instantaneous rate and a counter for the daily quota. Both live in Redis rather than in the worker, because Celery's own rate limit is per-worker and would silently multiply the request rate as workers scale out.

Every response is written to the raw zone before it is parsed. Failures are separated by kind: a transport error retries with exponential backoff and jitter; a rate limit retries after the limiter's own computed delay; an exhausted daily quota does *not* retry, because it cannot succeed again until the quota resets, and instead dead-letters immediately.

Transformation

Raw payloads are normalized into a source-independent record, validated per record with Pydantic and per batch with Pandera, then applied to the warehouse with change data capture. Two independent checks catch different failures: per-record validation catches a malformed price, while batch validation catches an entire source returning zero rows or a column that has become all-null after an upstream change.



The comparison is against the observation preceding this one in time, not the newest row, so backfilled history is judged correctly.

Figure 2. Insert-on-change with a heartbeat. Unchanged prices are not re-recorded, which keeps the event tables lean; the 24-hour heartbeat still records that the system looked and found no change.

Change data capture has two implementations with identical semantics. A row-by-row path is the readable reference; a set-based path stages the batch in a temporary table and expresses the same rules in SQL, so cost scales with batch size rather than record count. A parity test runs both over the same dataset and asserts identical output.

Loading

Product attributes are versioned as a Type 2 slowly changing dimension: when a tracked attribute changes, the open row is closed and a new one opened. Price observations are appended only when the price differs from the preceding observation, or when a 24-hour heartbeat has elapsed — which records that the system looked and found no change. History is never updated in place.

Section 8 — Analytics Layer

dbt Core on PostgreSQL, in three layers. Every downstream consumer — API, alerting, forecasting, dashboard and the written brief — reads marts, never the raw event tables. That is what keeps a business rule defined exactly once.

MART	QUESTION IT ANSWERS	ROWS
<code>mart_price_trend</code>	Daily series and day-over-day change; feeds forecasting	8,433
<code>mart_price_volatility</code>	Which SKUs move most, by coefficient of variation	272
<code>mart_price_gap_vs_own</code>	Competitor price versus our catalogue, matched on barcode	152
<code>mart_undercut_alerts</code>	Who is undercutting us, how badly, how fresh the evidence	108
<code>mart_out_of_stock_frequency</code>	Out-of-stock rate per SKU and retailer	0

Currency is part of the grain everywhere. The data spans 24 currencies. Subtracting a price in one currency from a price in another produces a number that looks authoritative and means nothing — and would fire false critical alerts, since 12 SEK beside 1.20 EUR reads as a catastrophic undercut when the two are close in value. The gap mart therefore joins on barcode *and* currency, so a cross-currency pair produces no row rather than a wrong one. Coefficient of variation is used as the volatility measure for the same reason: being unitless, it stays comparable across currencies and price levels.

Section 9 — Forecasting

Per-SKU short-horizon price forecasting using statistical models: AutoETS and AutoARIMA as candidates, with SeasonalNaive as a mandatory baseline. Prices are irregular — an observation appears when someone records it — so each series is resampled onto a daily grid and forward-filled, on the assumption that a shelf price holds until next observed.

Accuracy is measured by rolling-origin cross-validation, not by fit: the model is refitted on successive windows and scored on data it has not seen. Series shorter than a minimum length, or whose most recent observation is older than the staleness threshold, are excluded rather than forecast badly.

Backtested accuracy, latest run

MODEL	SERIES	AVG MAPE	WORST
AutoARIMA	9	3.13%	15.56%
SeasonalNaive (baseline)	9	3.13%	15.56%
AutoETS	9	3.13%	15.56%

All three models score identically, and the naive baseline is never beaten. This is reported rather than hidden: on short, sparse series retail prices behave close to a random walk, and a champion that cannot beat "tomorrow looks like today" is not earning its cost. The pipeline is correct; the data density is the limiting factor. A daily-refresh retail API is what would make forecasting genuinely informative here.

Section 10 — Product Matching

The same physical product appears under different barcodes, titles and languages across retailers. Matching them is what makes comparison possible beyond an exact barcode hit. The policy is deliberately ordered:

- 1. Structured identifiers win outright.** An exact barcode match is a fact; an embedding similarity is an opinion.
- 2. Blocking before vector search.** Candidates are restricted by category first, which cuts the comparison space and suppresses false positives on similar names.
- 3. Three confidence bands** rather than one cutoff.

BAND	RULE	ACTION	PAIRS
AUTO	cosine similarity ≥ 0.92	accepted automatically	370
REVIEW	0.80 to 0.92	queued for a human decision	2,187
REJECT	< 0.80	never surfaced	—

Embeddings are 384-dimensional, stored in PostgreSQL via pgvector with an HNSW index. A Streamlit screen presents candidate pairs side by side with their similarity score; each verdict is stored with reviewer and timestamp, forming the labelled set against which precision and recall can later be measured.

Section 11 — Serving and Visualization

API

ENDPOINT	RETURNS
GET /health	Liveness, database reachability, whether authentication is enabled
GET /products	Catalogue, filterable by category and tier
GET /prices/{product_id}	Daily price history with change percentage
GET /forecasts/{product_id}	Latest forecast with prediction intervals
GET /undercuts	Current undercuts, filterable by severity and staleness
GET /alerts	Raised alerts and delivery status
GET /matches/review	Candidate matches awaiting review

Alerting

Undercuts are evaluated every six hours. Each alert carries a severity derived from the size of the gap and a confidence label derived from the age of the evidence. Alerts are deduplicated so a persistent undercut is not re-sent every cycle, and alerts resting on evidence older than the freshness threshold are recorded but deliberately not delivered.

Dashboard and written brief

Metabase connects directly to the marts; all dashboard cards were executed against the live database. Separately, a weekly brief is generated from a closed set of facts queried from the marts. An optional LLM narration layer receives the same facts and its output is checked number by number against them; any figure not present in the facts causes the narrated version to be discarded in favour of the deterministic one.

Section 12 — Infrastructure

The target deployment is defined in Terraform: a VPC with the application host in a public subnet and the database in private subnets with no internet route, PostgreSQL on RDS with TLS enforced and no public address, an S3 bucket for the raw zone with versioning and lifecycle tiering, secrets in SSM Parameter Store, a least-privilege instance role, CloudWatch alarms, and a monthly budget alarm. Continuous integration runs linting, the test suites, dbt compilation and Terraform validation without requiring cloud credentials.

Not applied. The configuration passes `terraform fmt` and `terraform validate`, which confirms syntax and provider schema only. `terraform plan` has not been run against a real account, so a first apply should be expected to surface genuine issues such as availability-zone availability, engine version drift and IAM propagation delays.

Section 13 — Testing and Data Quality

SUITE	TESTS	COVERAGE
Python unit	~130	Normalizers, CDC rules, forecast metrics, alert logic, guardrail, authentication
Python integration	~20	Rate limiting against real Redis; CDC parity against a real database
dbt data tests	102	Schema, referential integrity, accepted ranges, cross-field invariants

The dbt tests assert relationships between fields rather than only per-column constraints: the computed gap must agree with its own inputs, the undercut flag must agree with the gap, maximum must not fall below minimum, and time must move forward within a series. A `not_null` test would catch none of these; each guards against a refactor silently inverting a rule.

Section 14 — Results

Delivered scale, measured with the pipeline idle and marts freshly built

MEASURE	VALUE
Price observations recorded	8,433
Observations in the last 30 days	3,736
Products tracked	3,932
Retailers observed	157
Monthly partitions in use	115
Undercuts detected	108
Undercuts on evidence 7 days old or less	62
Automated tests passing	252

Largest undercuts on current evidence

PRODUCT	RETAILER	OURS	THEIRS	GAP	AGE
Riz aux Champignons de Paris	U Express	2.23	1.59	-28.70%	6
Sriracha Hot Chilli Sauce	Super U	4.33	3.14	-27.48%	4
Tabasco Sauce Epicee Rouge	U Express	3.86	2.86	-25.91%	5
Sauce Quick Supreme Spicy	U Express	2.94	2.18	-25.85%	5
Amora Sauce Samourai 255g	E.Leclerc Express	2.16	1.65	-23.61%	6
Amora	U Express	1.57	1.20	-23.57%	6

Most volatile products by coefficient of variation

PRODUCT	RETAILER	MEAN	CV	BAND	POINTS
Petites Madeleines	E. Leclerc	2.43	0.455	high	5
Lentilles vertes	Centre Commercial E.Leclerc	1.21	0.375	high	7
PESTO ROSSO	E.Leclerc	2.46	0.370	high	5
PESTO ROSSO	Carrefour Market	2.34	0.352	high	6
Mais doux en grains	Centre Commercial E.Leclerc	0.50	0.345	high	8

Performance

OPERATION	RESULT
Replay of one stored day (947 payloads, 7,922 records)	41 s
The same work row-by-row over a 298 ms link	~3.3 h (est.)
Full warehouse rebuild from the raw zone	30 s
Embedding 1,251 products	35 s
Dashboard card query latency	141–486 ms

Section 15 — Challenges and Resolutions

CHALLENGE	DIAGNOSIS AND RESOLUTION
Forecast accuracy looked implausibly good	The first run reported 0.15% MAPE. The metric was scoring the forward-fill rather than the forecast: series are resampled onto a daily grid, and on a near-constant series predicting the last value is trivially correct. Scoring was changed to ignore filled days. Every model's reported accuracy became worse, which was the point.
Ingestion throughput collapsed against a remote database	The row-by-row CDC path issued roughly five statements per record — acceptable at sub-millisecond latency, pathological at 298 ms. A day's backfill would have taken about 3.3 hours. A set-based implementation reduced it to 41 seconds; both paths were kept, with a parity test asserting they produce identical output.
The warehouse described 2010, not the present	The source API returns oldest contributions first by default, so discovery filled the warehouse with genuine but decade-old observations — past every staleness gate, leaving nothing forecastable. Switching the ordering to most-recently-contributed changed the sample from 1 store on 1 date to 22 stores across 11 dates.
Product versions churned on every observation	The row-level product name is contributor-entered free text; one barcode carried 25 spellings, opening a new dimension version each time. The canonical catalogue name is used instead, with the free-text field as fallback.
An intermittent test failure	A rate-limiter test spent its burst then asserted the next call was refused — which silently depended on four Redis round-trips completing within one second. Under load they did not, a token refilled, and the test failed with nothing wrong. The refill rate was slowed so the assertion no longer races the clock.
Database lost to a storage fault	A Docker storage-layer corruption destroyed the volume mid-project. Because every raw payload had been written before parsing, the entire warehouse was rebuilt in 30 seconds with no upstream API calls. This validated the raw-first design under real failure rather than in theory.

Section 16 — Limitations and Future Work

Current limitations

1. **Availability tracking produces no data.** No connected source publishes stock levels. The table, partitions and mart are built against the real schema and begin producing numbers as soon as a stock-bearing source is connected.
2. **Forecasting is data-limited.** Only 11 products have series dense and recent enough to model, and the naive baseline is not beaten. This reflects the sparsity of crowd-sourced pricing, not a defect in the pipeline.
3. **Cloud infrastructure is unproven.** Validated but never applied.
4. **2,187 match candidates await review.** A genuine human workload; raising the review threshold would reduce it at the cost of recall.
5. **Cross-currency comparison is unsupported** by design, pending dated exchange rates.

Future work

1. **Connect a keyed retail source, starting with Best Buy.** Highest value per unit of effort: it is the simplest approval, it publishes stock — which alone activates availability tracking — and its refresh cadence supplies the dense daily

series forecasting currently lacks. Two limitations close together.

2. **Run a plan and apply against a real cloud account**, with a budget alarm configured before the first apply.
3. **Work the review queue to a few hundred labelled pairs**, then measure precision and recall and tune the threshold on evidence rather than intuition.
4. **Add dated exchange rates** to enable cross-currency comparison. The change is additive: a rates table, a converted column in the intermediate layer, and currency-agnostic marts alongside the existing scoped ones.
5. **Move to an orchestrator** such as Dagster or Airflow when backfill lineage across interdependent jobs becomes the operational pain point.

Appendix — Reproducing the results

```
docker compose up -d          # PostgreSQL + Redis
alembic upgrade head          # schema and partitions
python -m app.cli ingest openprices --limit 600
python -m app.cli seed openprices --tier 1
make dbt-build                 # models + 102 data tests
make forecast                  # backtest and forecast
make match                     # embed and match products
make alerts                    # evaluate undercuts
make brief                     # weekly brief
python -m app.cli status       # summary of the warehouse
```

Every figure in this report was read from the running system on 1 September 2026 with the pipeline idle and the marts freshly built. No value is estimated except where explicitly labelled as such.